

9. Replikation und Konsistenz

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2026

9.1 Vorschau

- Motivation
- Datenzentrierte Konsistenzmodelle
- Clientzentrierte Konsistenzmodelle
- Aktualisierungsarten
- Verteilung von Aktualisierungen

9.2 Motivation

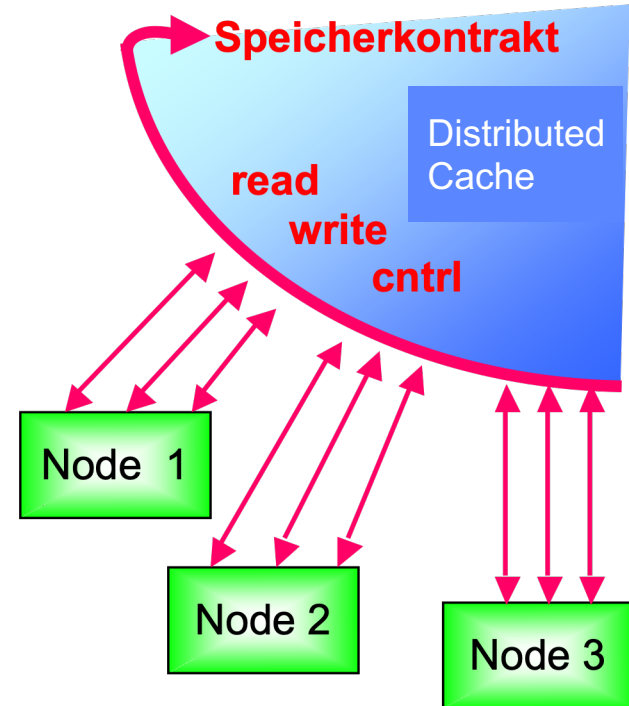
- Replikation: redundantes Vorhalten von Daten auf mehreren Servern
- Grund 1: Performanz
 - Lasterverteilung
 - lokaler Zugriff schneller
 - parallele Zugriffe auf viele Server simultan möglich
- Grund 2: Zuverlässigkeit
 - Redundanz erlaubt Maskierung von Serverausfällen

9.3 Datenzentrierte Konsistenzmodelle

- Für verteilte Caches / verteilter Storage
- Oder einen Distributed Shared Memory (DSM)
- Bem.: Datenobjekten werden durch eine globale eindeutige ID identifiziert

Konsistenzmodell

- Konsistenzmodell = Vertrag zwischen Programmen und Speicher
 - Speicherungsmechanismus garantiert den zugreifenden Prozessen Regeln nach welchen er die Schreiboperationen sichtbar werden lässt.



Konsistenzmodell

- Es existieren verschiedene Konsistenzmodelle
- Strenge Modelle: schränken Werte beim Lesezugriff ein
 - Einfach für den Programmierer
 - Langsamer durch häufige Synchronisierung
- Schwache Modelle: wenig Einschränkungen
 - Effizienter zu realisieren
 - Schwerer für den Programmierer

Konsistenz versus Kohärenz

- **Konsistenz:** Wann sieht ein anderer Server die Änderung?
- **Kohärenz:** Wie sieht ein anderer Server die Änderung?
→ Invalidierung (engl. invalidate) oder Aktualisierung (engl. update)

Notation für Beispiele

- **Schreib-Ereignisse** werden mit $W_{PID}(x)v$ bezeichnet:
 - PID = Prozess ID
 - v = Wert der geschrieben wird
 - x = Speicherstelle auf der geschrieben wird
- **Lese-Ereignisse** werden mit $R_{PID}(x)v$ bezeichnet
- Zeitpunkte der Ereignisse entspricht der physikalischen Zeit (Schreiben impliziert nicht das gleichzeitige Lesen)

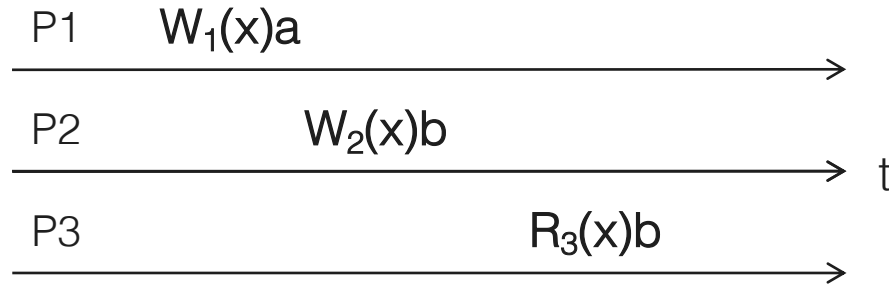


Strikte Konsistenz - Vertrag

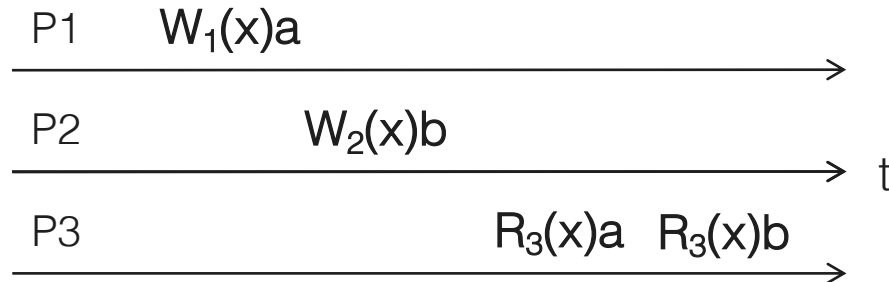
- Engl. strict consistency
- Definition: Alle Leseoperationen liefern den global „zuletzt“ geschriebenen Wert - unabhängig davon, welcher Prozess geschrieben hat.
- Problem: in einem Netz gibt es keine globale Zeit mehr. Der Begriff "zuletzt" ist unscharf.
- Es wird meist argumentiert, dass dieses Model im verteilten System nicht realisierbar ist, da Schreibzugriffe „sofort“ sichtbar sein müssten.

Strikte Konsistenz - Beispiel

- Strikt konsistent



- Nicht strikt konsistent



Linearisierbarkeit (engl. linearizability)

- Bei strikter Konsistenz wird unterstellt, dass es perfekt synchronisierte Uhren gibt, was nicht möglich ist. Daher wurde die Linearisierbarkeit eingeführt.
- Definition:
 - Wenn Operation A abgeschlossen ist, bevor Operation B beginnt, dann muss A auch logisch vor B stattfinden.
 - Überlappen sich zwei Operationen zeitlich, darf die Implementierung frei entscheiden, in welcher Reihenfolge sie logisch angeordnet werden.
- Literatur: M. Herlihy et.al., „Linearizability: A Correctness Condition for Concurrent Objects“, ACM Transactions on Programming, Languages and Systems, 1990.
- Hier wird das interaktiv sehr gut erklärt:
https://mwhittaker.github.io/consistency_in_distributed_systems/2_cap.html

Sequentielle Konsistenz - Vertrag

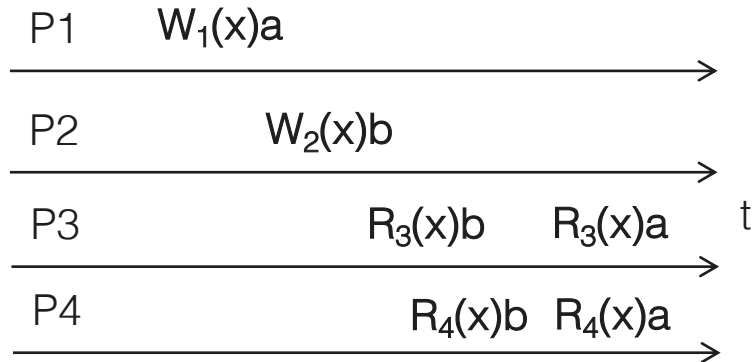
- Engl. sequential Consistency
- Definition: Das Ergebnis einer Ausführung ist dasselbe wie wenn alle Lese- und Schreib-Operationen aller Prozesse in einer bestimmten Reihenfolge ausgeführt werden.
 - Die Operationen eines Prozesses erfolgen dabei in der vom Programm definierten Reihenfolge.
 - Zugriffe verschiedener Prozesse sind zeitlich nicht sortiert

Bemerkungen

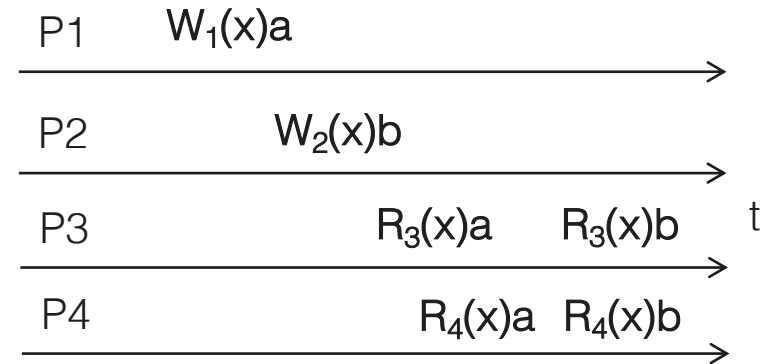
- Alle Prozesse sehen alle Schreiboperationen in gleicher Reihenfolge
- Im Gegensatz zur strikten Konsistenz, bleibt offen, wann eine Änderung sichtbar wird.
 - Evt. werden alte Daten verspätet gelesen
 - Evt. werden zuerst neue und dann alte Werte gelesen
 - Aber von allen Prozessen in der gleichen Reihenfolge
- Literatur:
 - L. Lamport, "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs", IEEE Transactions on Computers, 1979.

Sequentielle Konsistenz - Beispiele

- Beide Beispiele sind sequentiell konsistent, aber beide sind nicht strikt konsistent



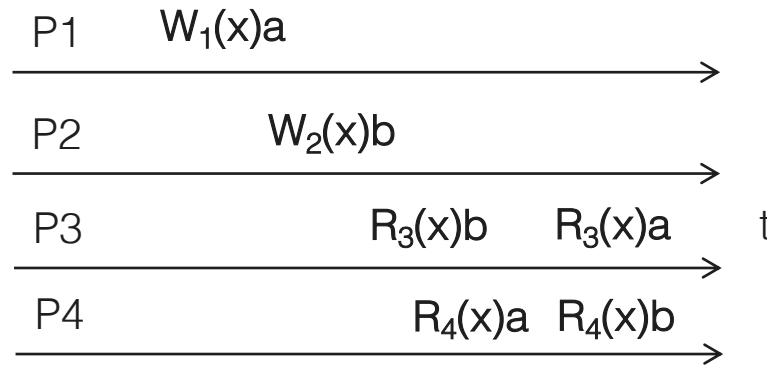
a)



b)

Sequentielle Konsistenz - Beispiele

- Dieses Beispiel ist weder sequentiell noch strikt konsistent
 - Im Prinzip reicht es festzustellen, dass es nicht sequentiell konsistent ist, damit ist es automatisch auch nicht strikt konsistent



c)

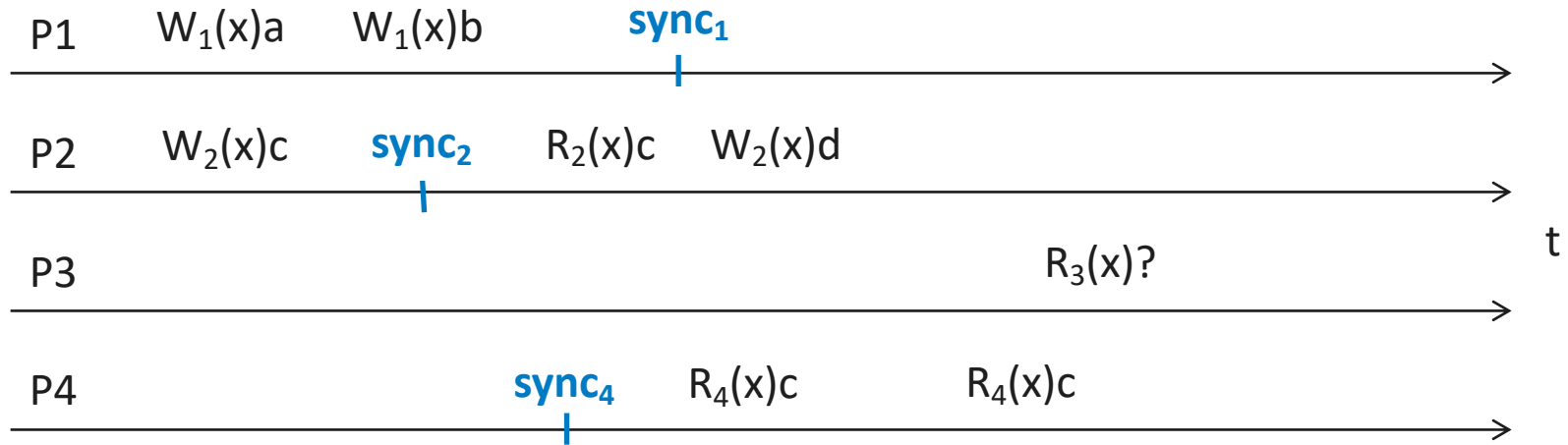
Sequentielle Konsistenz - Implementierung

- Schreiboperationen müssen bei allen Prozessen in gleicher Reihenfolge sichtbar werden (wann ist unwichtig)
- N Schreiber nebenläufig zu N Lesern
 - Schreiber schicken Änderungen per zuverlässigem total-geordneten Multicast.
 - Leser können veraltete Werte sehen (im Gegensatz zur strikten Konsistenz).

Schwache Konsistenz - Vertrag

- = weak consistency
- Es müssen explizit **sync()**-Aufrufe im Programm platziert werden
- Alle Aufrufe von **sync()** sind sequentiell konsistent
- Beim Aufruf von **sync()** werden alle vorherigen lokalen Schreiboperationen global propagiert.
- Ferner blockiert **sync()** so lange bis alle Schreiboperationen entfernter Server, die bei vorherigen **sync()**-Aufrufen sichtbar wurden, empfangen sind.

Schwache Konsistenz - Beispiel



Schwache Konsistenz - Implementierung

- Zentraler Server regelt Reihenfolge der `sync()`s
- Bei Aufruf von `sync()`:
 - Lokaler Datenspeicher verschickt pro Variablenobjekt seinen letzten Schreibwert (falls vorhanden) an den zentralen Server
 - Lokaler Datenspeicher erhält alle Aktualisierungen anderer Prozesse (die diese mit einem `sync()` gesendet haben) vom zentralen Server

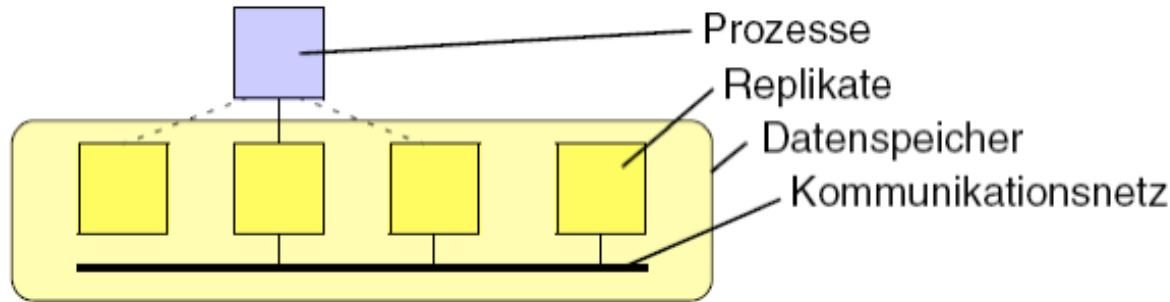
Schwache Konsistenz - Bemerkungen

- Vorteile:
 - Schreibzugriffe werden gebündelt synchronisiert
 - Viel Nebenläufigkeit möglich
- Nachteile:
 - `sync()`-Befehle müssen explizit und richtig „gesetzt“ werden
 - Prozess riskiert, veraltete Daten in seinem Speicher zu finden, wenn er nicht explizit zuvor seinen Speicher synchronisiert

9.4 Clientzentrierte Konsistenzmodelle

Allgemeines

- Bisher Konsistenz globaler Daten; für viele Anwendungen nicht notwendig
- Nun wird die Konsistenz aus der Sicht eines einzelnen Clients betrachtet
 - Modelle garantieren Ordnung von Operationen für einzelne Klienten.



Eventual Consistency

- Vertrag: Daten werden letztendlich konsistent.
- Push-Modell: Daten werden periodisch an alle Replikate gesandt
- Pull-Modell: Replikate holen sich bei Bedarf oder periodisch ein Update
- Weit verbreitetes Konsistenzmodell bei Cloud-Diensten /-Anwendungen

Monotones Lesen

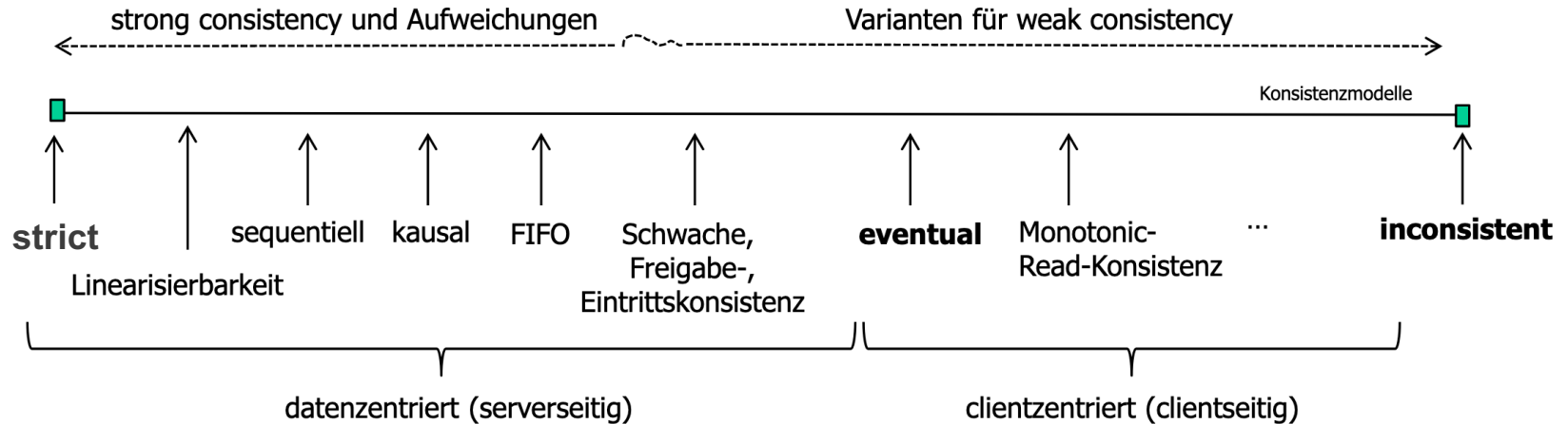
- Vertrag: Falls ein Prozess einen Wert a aus Datum x von einem Replikat gelesen hat, dann liest er zukünftig weiterhin a oder einen neueren Wert.
- Es kann also kein veralteter Wert gelesen werden
- Anwendungsbeispiel: replizierte Mailbox / Kalender
 - Der Client kann von jedem Replikat/Store lesen
 - Siehe auch nächste Seite

Monotones Lesen - Implementierung

- Schreibzugriffe mit global eindeutigem Identifikator versehen
- Jeder Client speichert einen Read-Set
 - Read-Set speichert Identifikatoren der gelesenen Schreibzugriffe
- Lesezugriff (Client schickt seinen Read-Set mit)
 - Das kontaktierte Replikat vergleicht Read-Set mit eigenen Aktualisierungen
 - Nur wenn das Replikat alle Schreibzugriffe kennt, die im Read-Set des Clients sind, nur dann ist der Lesezugriff erlaubt, ansonsten ist das Replikat veraltet
 - Falls der Zugriff erlaubt ist erhält der Client einen Identifikator für den neueren Wert, wodurch sein Read-Set erweitert wird

Zusammenfassung - Konsistenzmodelle

- Je strenger, desto aktueller ist der Speicher, aber evt. langsamer
- Je schwächer, desto schneller das System, aber Speicher nicht immer aktuell



Bild, Prof. Dr. Peter Mandl, TU München

9.5 Aktualisierungsarten

- Die Implementierungsalternativen für die Art der Aktualisierung ist unabhängig vom Konsistenzmodell
- Art der Aktualisierung:
 - Verteilung von Aktualisierungshinweisen (engl. invalidate)
 - Verteilung der aktualisierten Werte (engl. update)
 - Verteilung der aktualisierenden Operation

Aktualisierungshinweise

- Lokaler Datenspeicher erfährt, dass eine Aktualisierung stattfand
- Z.B. durch eine Invalidierungsnachricht (engl. invalidate)
- Führt zur Löschung betroffener Daten
- Beim nächsten Lesezugriff müssen Daten neu „geholt“ werden
→ Pull-Modell

Aktualisierungshinweise - Bewertung

- Vorteil:
 - Invalidierungen benötigen wenig Kommunikationsbandbreite
 - Mehrere entfernte Aktualisierungen werden nur einmal geholt
 - Mehrfache Invalidierungen können unterbleiben (z.B. durch Führen einer Liste von noch gültigen Replikaten)
- Nachteil:
 - Bei häufigem Lesen auf invalidierten Daten müssen diese jedes Mal erst beschafft werden
 - Hier wäre eine direkte Aktualisierung effizienter
 - Aber dies ist abhängig vom Zugriffsmuster welches vorab unbekannt ist und sich u.U. dynamisch ändert

Werte aktualisieren

- Lokaler Datenspeicher erfährt die neuen Werte (update) → Push-Modell
- Vorteil:
 - Lokale Daten sind immer aktuell
→ schnelle Lesezugriffe, ohne Netzwerkkommunikation
- Nachteil:
 - Update-Nachrichten benötigen mehr Bandbreite als Invalidate-Hinweise
 - Werden Daten lange nicht gelesen werden diese unnötig aktualisiert
 - U.U. viele unbenutzte Replikate → Speicherverschwendung

Aktualisierende Operationen

- Lokaler Datenspeicher erfährt Operation und Parameter
 - Führt diese auf dem lokalen Datenspeicher durch
 - Vermeidet aufwändige Datenübertragung, insbesondere bei „BigData“ bevorzugt
- Vorteil:
 - Operationsname und Parameter oft kleiner als veränderte Daten
 - In diesem Fall genügt eine geringere Netzwerkbandbreite
- Nachteil:
 - Lokale Operationsausführungen müssen gleiche Ergebnisse liefern
 - Es sollten alle Daten auf einem Rechner vorhanden sein, auf dem die Operation durchgeführt wird.

9.6 Verteilung von Aktualisierungen

Primärkopieverfahren

- Ausgezeichnetes Replikat koordiniert und verteilt die Schreibzugriffe auf alle anderen Replikate → Primärkopie (engl. primary replica)
- Kandidaten für eine Primärkopie
 - Erzeuger (engl. creator) eines Datenobjektes
 - Oder der aktuelle Eigner (engl. owner), falls die Primärkopie dynamisch wechselt
- Beispiel:
 - Primary-Replica bei Google File System, siehe später

Auffinden der Primärkopie

- Über einen zentralen Server, z.B. Google File System
- Oder verteilten Verzeichnisdienst, z.B. Distributed Hashtable (→ P2P)
- Durch einen Multicast an alle Server

Quorenbasierte Replikation

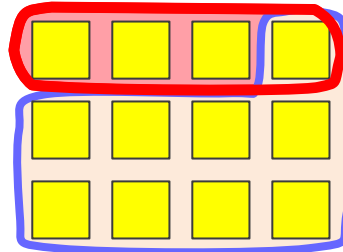
- Ausführung der Operationen auf einigen, aber evt. nicht allen Replikaten
- Lesezugriff: Lesen von N_R Replikaten
 - N_R = Lese-Quorum
- Schreibzugriff: Schreiben auf N_W Replikaten
 - N_W = Schreib-Quorum

Quorenbasierte Replikation

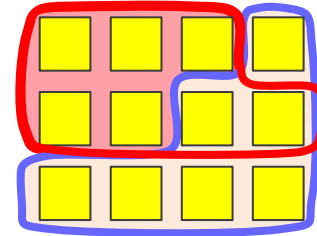
- Wahl von N_R und N_W so, dass beide nachfolgenden Bedingungen gelten.
- 1. Bedingung: $N_R + N_W > N$
 - Lese-Schreib-Zugriffe überlappen in mindestens einem Replikat
 - Beim Lesen hat dadurch mindestens ein Replikat die neueste Version
- 2. Bedingung: $N_W > N/2$:
 - Zwei Schreib-Zugriffe überlappen in mindestens einem Replikat
 - Wenigstens ein Replikat erhält Kenntnis über beide Schreib-Operationen

Quorenbasierte Replikation

- Beispiele: $N=12$

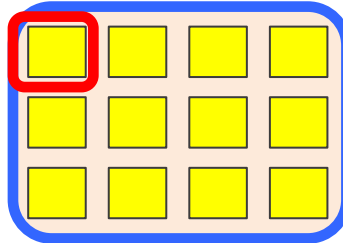


$$N_R=4, N_W=9$$



$$N_R=7, N_W=7$$

- Spezialfall: Read-One, Write-All (ROWA)



$$N_R=1, N_W=12$$

Bewertung

- Es müssen nicht immer alle Replikate erreicht werden
→ der Ausfall von einem oder mehreren Servern kann maskiert werden
- Je nach Konfiguration und Zugriffsmuster weniger oder mehr Kommunikation
- Konflikte müssen erkannt und aufgelöst werden
→ z.B. durch Zeitstempel oder Sequenznummern

9.7 Zusammenfassung

Zusammenfassung

- In verteilten Systemen wird immer repliziert
→ Wegen Performance & Fehlertoleranz
- Die Konsistenzfrage stellt sich unmittelbar, sobald replizierte Daten beschrieben werden dürfen, also auch fast immer
- Für die Praxis sind Strenge Konsistenz & Eventual Consistency wichtig
 - Je strenger die Konsistenz, desto einfacher zu programmieren, aber eingeschränkte Skalierbarkeit, da häufiger synchronisiert wird
 - Es gibt zahlreiche Konsistenzmodelle ...